Retrieval-Augmented Generation

Session 5 · AI Integration & RAG · Ejo Labs University Track · 11 August 2026

RAG = find the relevant pages, paste them into the prompt, then ask.

Not a model. Not a library. Not a product. It is an architecture — a pattern for what you put in the prompt.

THE THREE WORDS

Retrieval search your own text, pull back the closest pieces. No AI writing happens here.

Augmented those pieces go into the prompt beside the question. The prompt gains facts the model never had.

Generation the model reads that prompt and writes the answer.

THREE WAYS TO GIVE IT A DOCUMENT

All three are the same thing. "Attaching a file" means reading the file and putting the text in the prompt. Nothing is uploaded.

plain text paste the document straight into the prompt string.

JSON the same text with labels, so the model can say which document it used. Useful once you send more than one.

from a file open(), read, then exactly the same as plain text.

This already works. It stops working when documents get bigger: it stops fitting, it costs more every request, and burying one useful line in forty pages makes the answer worse. So: send only the part that answers the question.

THE FOUR-PART PROMPT

```
1. INSTRUCTION  what to do, and what to
                do when you can't
2. CONTEXT      retrieved pieces, numbered,
                each with its source
3. QUESTION     the user's words, unmodified
4. OUTPUT RULE  language, length, citation
```

Question last. Number the pieces and label the source so the model can cite and you can debug. Never reword the user's question — you throw away the words retrieval matched on.

```
Use ONLY the context below to answer.
If the answer is not in the context, say
you don't know. Do not invent anything.
Answer in one or two sentences, and end
with the number of the context item you
used, like [1].

CONTEXT:
[1] (kanombe-clinic.md · Emergency contact)
    If you need help at the weekend, at
    night, or on a public holiday, call the
    emergency number: 0788 123 456.

QUESTION: What number do I call at the
weekend?
```

"If the answer is not in the context, say you don't know." Without that sentence the model fills the silence. It is the highest-leverage line in the whole prompt.

THE FOUR KNOBS YOU TUNE

chunk size ~500 characters to start. Too small is a fragment with no context; too big buries one useful line among nine.

overlap ~50 characters. Repeat the tail of each piece at the head of the next, so a fact on a boundary survives.

top-k how many pieces you paste in. More context costs money and can make answers worse, not better.

embedding model the only part that decides whether your language is understood. Measure it; do not trust a language list.

THE EMBEDDING TRAP

Three facts, each written twice in different words. Our baseline put each pair together — it looked like it understood meaning. Then we deleted every digit, which changes no meaning at all:

	with digits	without	change
phone	0.260	0.114	-56%
fees	0.298	0.216	-28%
jabs	0.268	0.189	-29%
the gap	0.138	0.046	-66%

Two thirds of the "understanding" was the same phone number appearing in both halves of a pair. When something looks like it works, find out why before you believe it.

VOCABULARY

embedding a list of numbers standing for the meaning of a text. Not a summary, not reversible, not the LLM.

dimension how many numbers are in that list. 384 and 768 are common.

cosine similarity dot(a,b) divided by norm(a)·norm(b). –1 to 1; 1 means the two texts point the same direction in meaning.

chunk one retrievable piece of a document, with its metadata.

grounding answering from supplied context rather than from memory.

recall@k of your labelled questions, the fraction where the right piece appeared in the top k. The metric that matters.

ANN index HNSW and friends — makes lookup sub-linear so millions of pieces stay searchable.

re-ranking a second, slower model reorders the top k. Beyond today.

EJOCHAT — EJOLABS.COM/EN/API

```
POST https://api.ejolabs.com/api/v1/subiza
X-API-Key: kgpt_...
{"messages":[{"role":"user",
              "content":"..."}]}
```

messages is a list because the server has no memory — you resend the conversation every turn. Free tier 50 requests/day. The key lives in .env: never in code, never in git, never in front-end JavaScript.

400	body does not match the API
401	key missing or wrong
402	trial over / billing inactive
403	key not allowed here
422	missing field or limit exceeded
429	quota gone — 50/day on free
503	upstream down
5xx	their side

Retry 429 and 5xx with exponential backoff. Never retry 401, 403 or 422 — that is only being wrong faster.

FOUR WAYS THIS STILL ANSWERS WRONGLY

1. Retrieval returned the wrong piece — confident, grounded and cited, which is more convincing than a guess and so more dangerous.
2. The answer is not in your documents, and you did not allow "I don't know".
3. The documents are out of date. RAG serves them with confidence.
4. Retrieval worked and the model ignored it.

Always show which piece the answer came from. One line of code, and it is how your user catches you being wrong.

And if the answer is one row in a table, run the query. RAG is for prose, not for SELECT * FROM users WHERE id = 7.

COMMANDS

```
python setup_check.py
python 01_ask_without_rag.py  # it lies
python 02_add_context.py      # it works
python 05_chunking.py         # the seam
python 06_embeddings.py       # the trap
python 08_rag.py              # the prompt
python 11_evaluate.py         # the truth

# no key and no internet needed
EJO_OFFLINE=1  EJO_EMBEDDER=hashing
```

HOMEWORK — BEFORE SESSION 7, 18 AUG

Point the pipeline at 3–5 real documents from your own capstone. Write 10–15 labelled questions BEFORE you measure. Submit one question it answers correctly and one it answers wrongly, with sources and scores — then two sentences on why the wrong one was wrong.

Those two sentences are 35% of the mark. Not "it hallucinated": which stage failed — chunking, retrieval, the prompt, or the documents — and how you know.